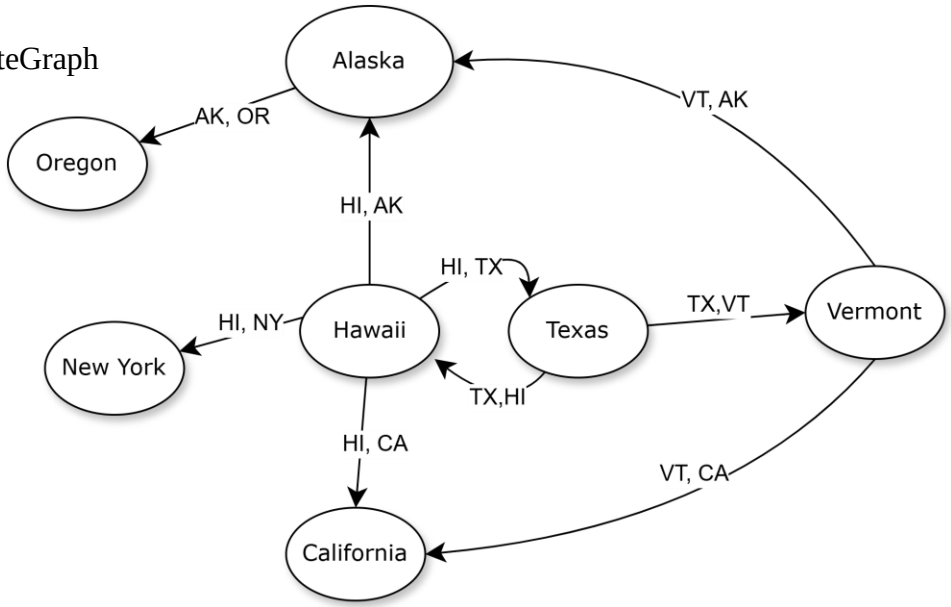
CMSC204
Kartchner

V(StateGraph) = {Oregon, Alaska, Texas, Hawaii, Vermont, NewYork, California}
E(StateGraph) = {(Alaska, Oregon), (Hawaii, Alaska), (Hawaii, Texas), (Texas, Hawaii), (Hawaii, California), (Hawaii, New York), (Texas, Vermont), (Vermont, California), (Vermont, Alaska)}

1. Draw the StateGraph



1. Describe the graph pictured above, using the formal graph notation.

   V(StateGraph) = {OR. AK. TX, HI, VT, NY, CA}

   E(StateGraph) = {{AK,OR},{HI,AK},{HI,NY},{HI,TX},{TX,HI},{HI,CA},{TX,VT}, {VT,CA,},{VT,AK}}

2. a. Is there a path from Oregon to any other state in the graph? NO

   b. Is there a path from Hawaii to every other state in the graph? YES

   c. From which state(s) in the graph is there a path to Hawaii? Texas

3. a. Show the adjacency matrix that would describe the edges in the graph.
Store the vertices in alphabetical order

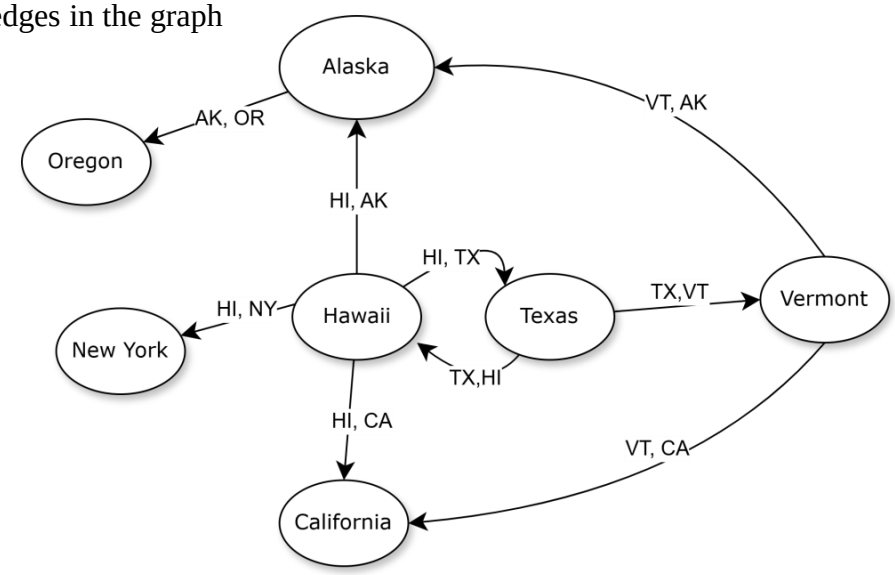
States

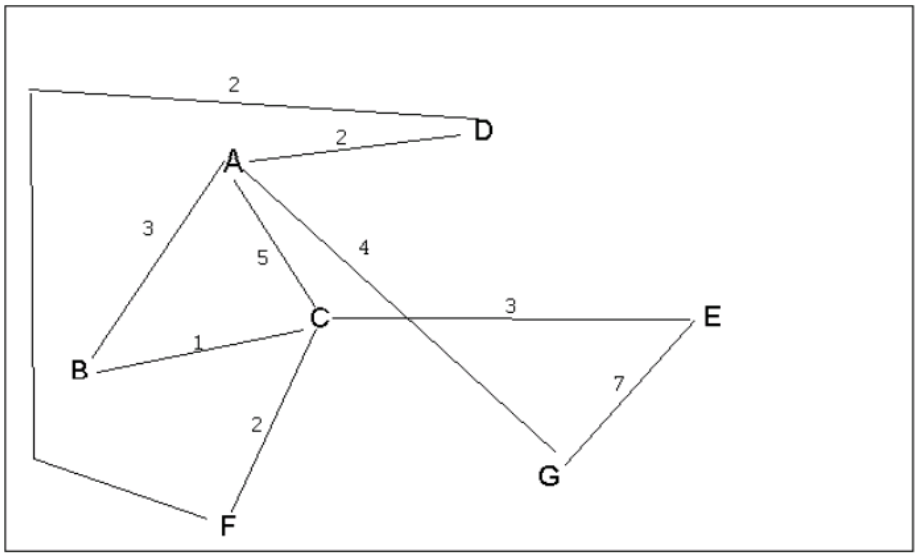
$A =$

	0	0	0	0	0	0	0
AK	1	0	0	0	0	0	0
TX	0	0	0	1	1	0	0
HI	0	1	1	0	0	1	1
VT	0	1	0	0	0	0	1
NY	0	0	0	0	0	0	0
CA	0	0	0	0	0	0	0
	OR	AK	TX	HI	VT	NY	CA

3. b. Show the adjacency lists
that would describe the edges in the graph

OR:
AK: OR
TX: HI, VT
HI: TX, CA, NY, AK
VT: AK, CA
NY:
CA:



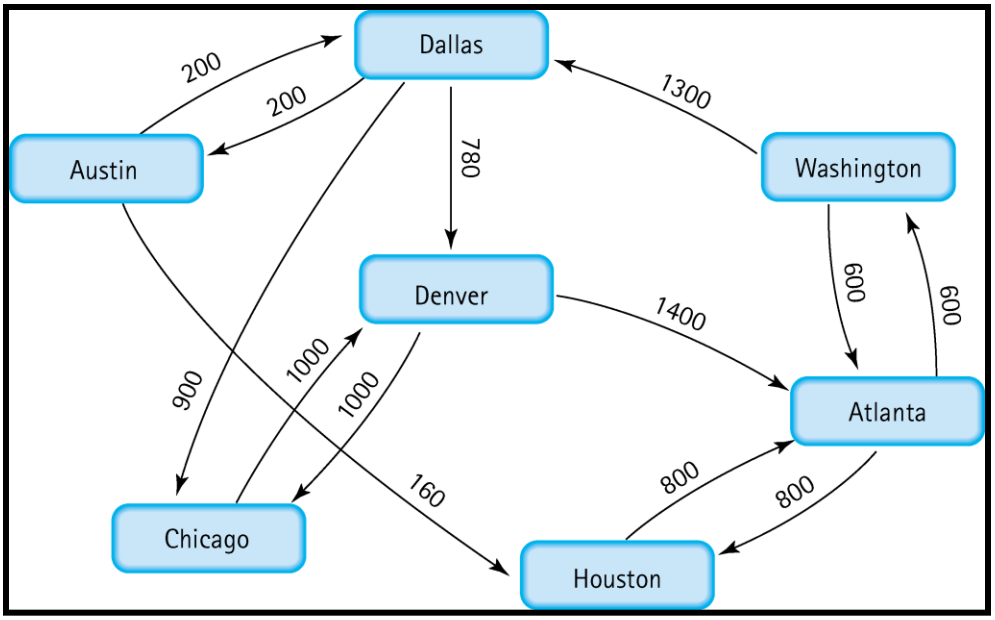


4 a. Which of the following lists the graph nodes in depth first order beginning with E?

A) E, G, F, C, D, B, A
B) G, A, E, C, B, F, D
C) E, G, A, D, F, C, B
D) E, C, F, B, A, D, G

4 b. Which of the following lists the graph nodes in breadth first order beginning at F?

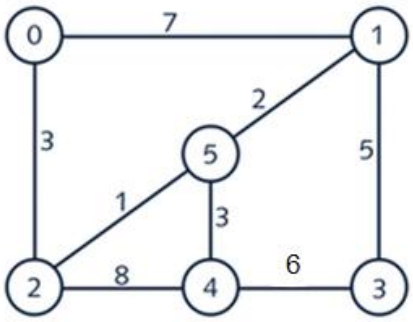
A) F, C, D, A, B, E, G
B) F, D, C, A, B, C, G
C) F, C, D, B, G, A, E
D) a, b, and c are all breadth first traversals



5. Find the shortest <u>distance</u> from Atlanta to every other city

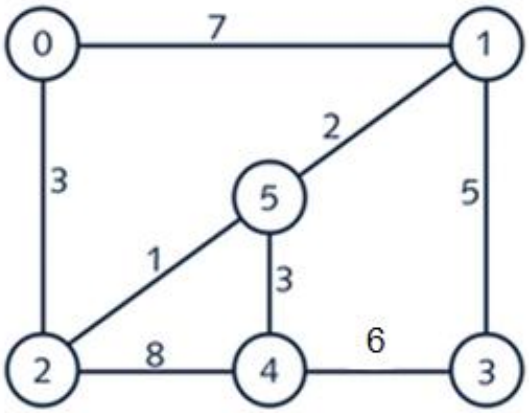
Atlanta to Atlanta = 0
Atlanta to Washington = 600
Atlanta to Houston = 800
Atlanta to Denver = 960
Atlanta to Dallas = 1300
Atlanta to Austin = 1500
Atlanta to Chicago = 1960

6. Find the minimal spanning tree using Prim's algorithm. Use 0 as the source vertex . Show the steps.



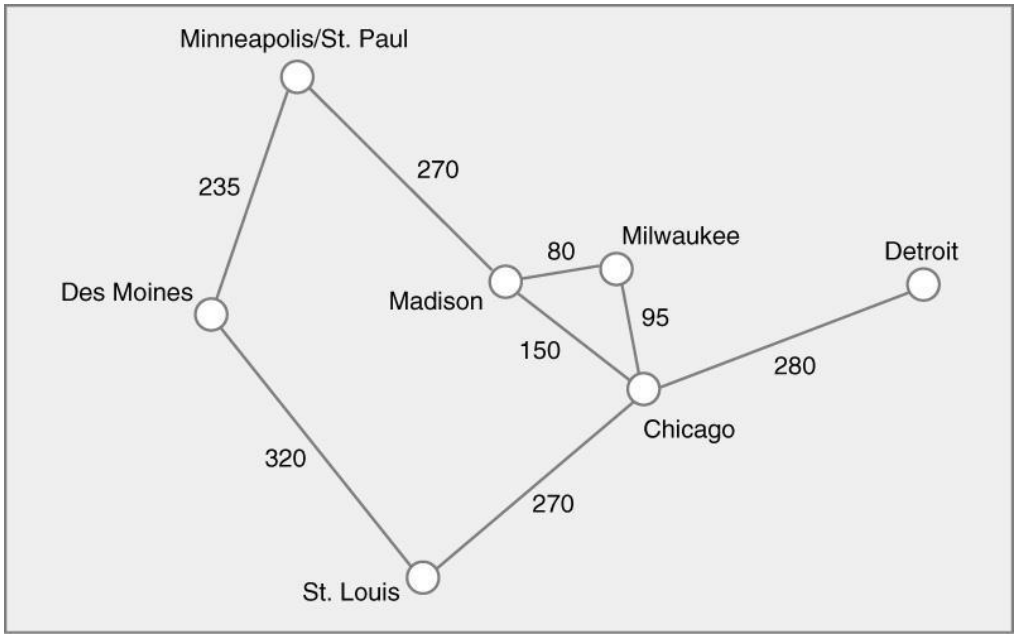
	Select cheapest:	**Visited**	**MST edges**	**Candidate edges**
Steps	Start: 0	{0}	[ ]	{(0,2, weight = 3), (0,1, weight =7)}
1	(0,2) =3	{0, 2}	[(0,2,3)]	add(2,5, weight = 1), remove(0,2), (0,1, weight =7), add(2,4, weight = 8)}
2	(2,5) =1	{0,2,5}	[(0,2,3), (2,5,1)]	{remove(2,5), add(5,1,weight=2), (0,1, weight =7), (2,4, weight = 8)}
3	(5,1) =2	{0,2,5,1}	[(0,2,3), (2,5,1), (5,1,2)]	{remove(5,1), add(5,4, weight=3), add(1,3,weight=5), ignore(0,1, weight =7), (2,4, weight = 8)}
4	(5,4) =3	{0,2,5,1,4}	[(0,2,3), (2,5,1), (5,1,2), (5,4,3)]	{ (1,3, weight=5), add(3,4, weight=6), remove(5,4), remove(2,4)}
5	(1,3) =5	{0,2,5,1,4, 3}	[(0,2,3), (2,5,1), (5,1,2), (5,4,3), (1,3,5)]	{ remove(1,3), remove(3,4)}
Done	**Total** weight of MST: 3+1+2+3+5 **= 14**	all vertices are Visited	Edges: (0,2), (2,5), (5,1), (5,4), (1,3)	

7. Find the minimal spanning tree using Kruskal's algorithm. Show the weights in order and the steps.



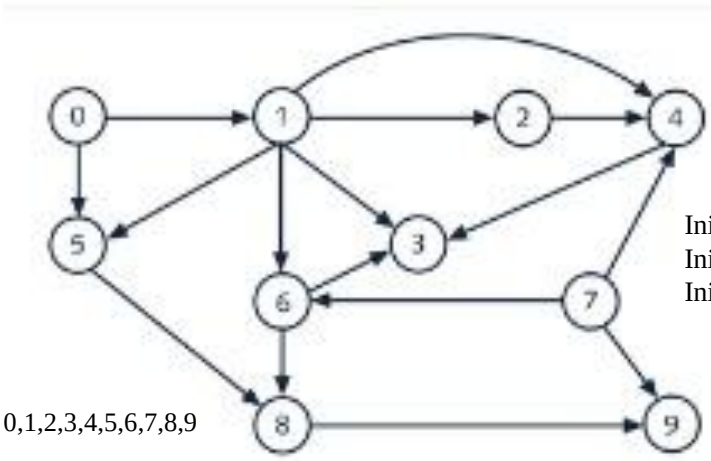
	Add edge:	**sets**	**MST edges**	
Steps	Start: 0	{0}	[ ]	
1	(2,5) =1	{0}, {1}, {2,5}, {3}, {4}	{(2,5)}	Does **not** create cycle
2	(1,5) =2	{0}, {1,2,5}, {3}, {4}	{(2,5), (1,5)}	Does **not** connect vertices already in the set (1,5)
3	(0,2)=3	{0,1,2,5}, {3}, {4}	{(2,5), (1,5), (0,2)}	Does **not** connect vertices already in the set (0,2)
4	(4,5) =3	{0,1,2,4,5}, {3}	{(2,5), (1,5), (0,2), (4,5)}	Does **not** connect vertices already in the set (4,5)
5	(1,3) =5	{0,1,2,4,5, 3}	{(2,5), (1,5), (0,2), (4,5), (1,3)}	Does **not** connect vertices already in the set (1,3)
Done	**Total** weight of MST: 1+2+3+3+5 **= 14**			

8. Find the minimal spanning tree using the algorithm you prefer. Use Minneapolis/St. Paul as the source vertex



	Extract minimum weight edge from priorityQueue	Visited	MST edges	priorityQueue (edge : weight)
Steps	Start: MSP	{MSP}	[ ]	{(MSP, DM): 235), (MSP, Mad): 270}
1	(MSP, DM): 235	{MSP, DM}	[(MSP, DM, 235)]	{(MSP, Mad): 270), (DM, STL) : 320}
2	(MSP, Mad): 270	{MSP, DM, Mad}	[(MSP, DM, 235), (MSP, Mad, 270)]	{(Mad, Mil): 80, (Mad, Chi): 150, (DM, STL): 320}
3	(Mad, Mil): 80	{MSP, DM, Mad, Mil}	[(MSP, DM, 235), (MSP, Mad, 270), (Mad, Mil, 80)]	{(Mil, Chi): 95, (Mad, Chi): 150, (DM, STL): 320}
4	(Mil, Chi): 95	{MSP, DM, Mad, Mil, Chi}	[(MSP, DM, 235), (MSP, Mad, 270), (Mad, Mil, 80), (Mil, Chi, 95)]	{(Mad, Chi): 150, (Chi, STL): 270, (Chi, Det): 280, (DM, STL): 320}
5	(Mad, Chi): 150 -> Discard (Chi visited) (Chi, STL): 270	{MSP, DM, Mad, Mil, Chi, STL}	[(MSP, DM, 235), (MSP, Mad, 270), (Mad, Mil, 80), (Mil, Chi, 95), (Chi, STL, 270)]	{(Chi, Det): 280, (DM, STL): 320}
Done	(Chi, Det): 280 (DM, STL): 320 -> Discard (STL visited)	{MSP, DM, Mad, Mil, Chi, STL, Det}	Final Edges: (MSP, DM, 235) (MSP, Mad, 270) (Mad, Mil, 80) (Mil, Chi, 95) (Chi, STL, 270) (Chi, Det, 280)	[ ] (Empty or only edges to visited nodes)
	Total weight of MST: 235 + 270 + 80 + 95 + 270 + 280 = **1230**	all vertices are Visited		

9. List the nodes of the graph in a breadth first topological ordering. Show the steps using arrays predCount, topologicalOrder and a queue



Initialize Queue: 7 has predCount 0
Initialize topologicalOrder array: [ ]
Initialize orderIndex: 0

Vertices: 0,1,2,3,4,5,6,7,8,9
(V= 10)

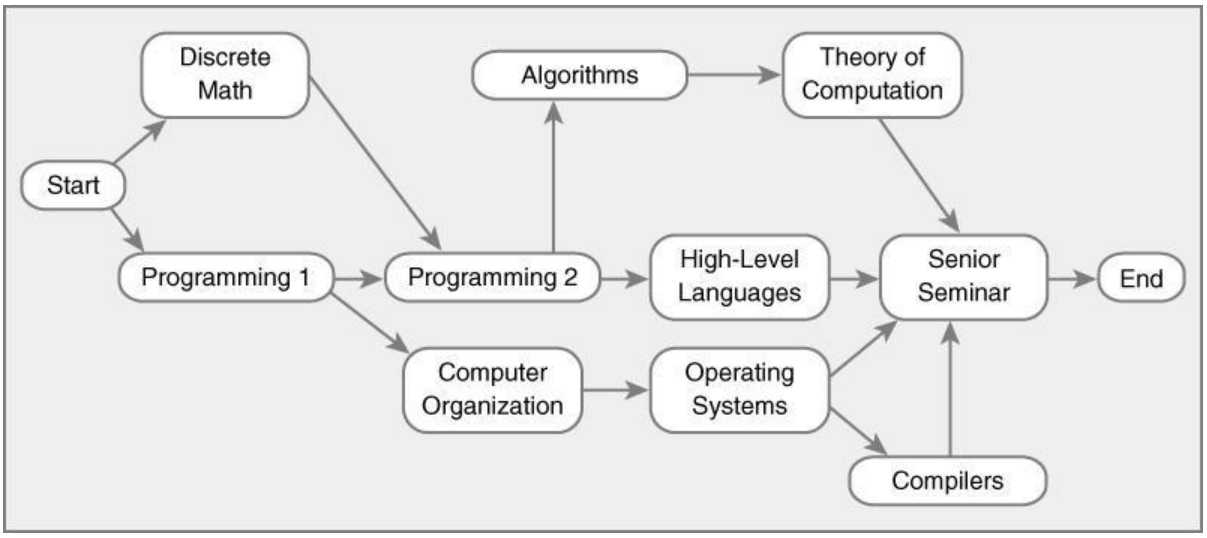
Edges:
- 0 -> 1, 0 -> 5
- 1 -> 2, 1 -> 4, 1 -> 6
- 2 -> 4
- 3 -> 6
- 4 -> 3
- 5 -> 6, 5 -> 8
- 6 -> 8
- 7 -> 4, 7 -> 9
- 8 -> 9

Initial predCount (In-degree):
predCount = 0
predCount = 1 =1 (from 0)
predCount = 2 =1 (from 1)
predCount = 3 = 1 (from 4)
predCount = 4 = 3 (from 1,2,7)
predCount = 5 = 1 (from 0)
predCount = 6 = 3 (from 1,3,5)
predCount = 7 = 0
predCount = 8 = 2 (from 5,6)
predCount = 9 =2 (from 7,8)

FINAL TOPOLOGICAL ORDER:
[7] [1][5][2][4][3][6][8][9]

Step	Action	Queue	predCount Array	topologicalOrder Array (Up to orderIndex)
0	Initial	[7]	[1][1][1][3][1][3][2][2]	[]
1	Dequeue 0	[7]	[1][1][1][3][1][3][2][2]	``
	Neighbors 1,5	[7][1][5]	[1][1][3][3][2][2]	``
2	Dequeue 7	[1][5]	[1][1][3][3][2][2]	[7]
	Neighbors 4,9	[1][5]	[1][1][2][3][2][1]	[7]
3	Dequeue 1	[5]	[1][1][2][3][2][1]	[7][1]
	Neighbors 2,4,6	[5][2]	[1][1][2][2][1]	[7][1]
4	Dequeue 5	[2]	[1][1][2][2][1]	[7][1][5]
	Neighbors 6,8	[2]	[1][1][1][1][1]	[7][1][5]
5	Dequeue 2	[]	[1][1][1][1][1]	[7][1][5][2]
	Neighbor 4	[4]	[1][1][1][1]	[7][1][5][2]
6	Dequeue 4	[]	[1][1][1][1]	[7][1][5][2][4]
	Neighbor 3	[3]	[1][1][1]	[7][1][5][2][4]
7	Dequeue 3	[]	[1][1][1]	[7][1][5][2][4][3]
	Neighbor 6	[6]	[1][1]	[7][1][5][2][4][3]
8	Dequeue 6	[]	[1][1]	[7][1][5][2][4][3][6]
	Neighbor 8	[8]	[1]	[7][1][5][2][4][3][6]
9	Dequeue 8	[]	[1]	[7][1][5][2][4][3][6][8]
	Neighbor 9	[9]	``	[7][1][5][2][4][3][6][8]
10	Dequeue 9	[]	``	[7][1][5][2][4][3][6][8] [9]
	No Neighbors	[]	``	[7][1][5][2][4][3][6][8] [9]

10. List the nodes of the graph in a breadth first topological ordering.



Start, Discrete Math, Programming 1, Programming 2, Computer Organization, Algorithms, High-Level Languages, Operating Systems, Theory of Computation, Compilers, Senior Seminar, End